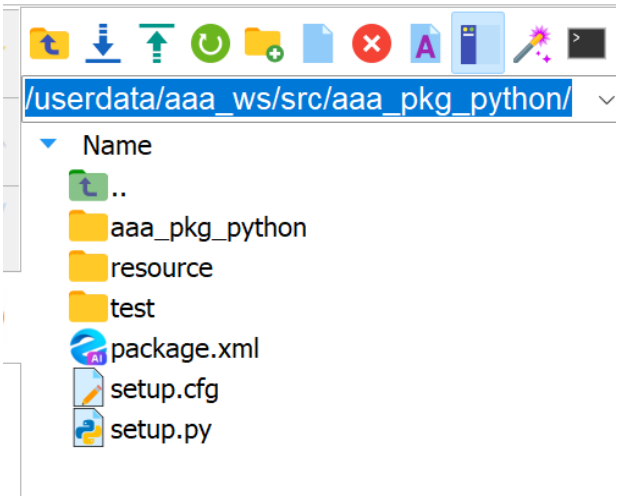
实验三. ROS2 基本编程

实验目的

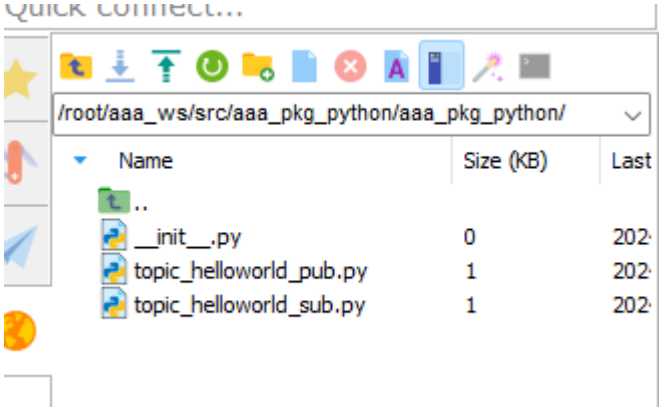
1）熟悉 ROS2 文件架构，工作空间、功能包、节点创建和编译等
2）了解 ROS 运行方式 ros2 run 和 ros2 launch

实验过程

1）创建工作空间、功能包
同组学员工作空间分别是 aaa_ws、bbb_ws、ccc_ws。下面操作以 aaa 为例，bbb 和 ccc 相应操作。
在工作目录（/userdata），创建工作空间，输入
cd /userdata
mkdir aaa_ws/src -p
进入 src 目录
cd aaa_ws/src
输入命令：
ros2 pkg create --build-type ament_python aaa_pkg_python
aaa_pkg_python 功能包创建好了，观察文件夹结构



2）创建节点并编辑
把源程序 topic_helloworld_pub.py 和 topic_helloworld_sub.py 上传到 aaa_ws/src/aaa_pkg_python/aaa_pkg_python 下



修改 aaa_ws/src/aaa_pkg_python/setup.py 文件

增加两个节点

```
    tests_require=['pytest'],
    entry_points={
        'console_scripts': [
        'pub=aaa_pkg_python.topic_helloworld_pub:main',
        'sub=aaa_pkg_python.topic_helloworld_sub:main',
        ],
    },
)
```

在 aaa_ws 目录下，进行编辑

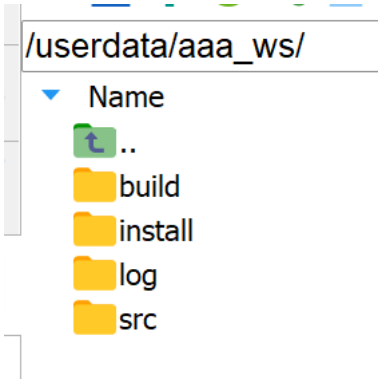
cd ..

colcon build

```
root@ubuntu:/userdata/aaa_ws/src# cd ..
root@ubuntu:/userdata/aaa_ws# colcon build
Starting >>> aaa_pkg_python
Finished <<< aaa_pkg_python [6.38s]

Summary: 1 package finished [9.58s]
root@ubuntu:/userdata/aaa_ws#
```

观察 aaa_ws 目录下的文件夹



3）运行节点

设置路径（每开一个窗口都要设置一下），在 aaa_ws 目录下，输入：

source install/setup.bash

运行第一个节点

ros2 run aaa_pkg_python pub

```
root@ubuntu:~/aaa_ws# source install/setup.bash
root@ubuntu:~/aaa_ws# ros2 run aaa_pkg_python pub
[INFO] [1722172956.702547095] [topic_helloworld_pub]: Publishing: "0_Hello World"
[INFO] [1722172957.125721362] [topic_helloworld_pub]: Publishing: "1_Hello World"
[INFO] [1722172957.625671891] [topic_helloworld_pub]: Publishing: "2_Hello World"
[INFO] [1722172958.125719892] [topic_helloworld_pub]: Publishing: "3_Hello World"
[INFO] [1722172958.625804258] [topic_helloworld_pub]: Publishing: "4_Hello World"
[INFO] [1722172959.125716865] [topic_helloworld_pub]: Publishing: "5_Hello World"
[INFO] [1722172959.625768601] [topic_helloworld_pub]: Publishing: "6_Hello World"
[INFO] [1722172960.125738719] [topic_helloworld_pub]: Publishing: "7_Hello World"
[INFO] [1722172960.625667881] [topic_helloworld_pub]: Publishing: "8_Hello World"
[INFO] [1722172961.125704222] [topic_helloworld_pub]: Publishing: "9_Hello World"
[INFO] [1722172961.625667821] [topic_helloworld_pub]: Publishing: "10_Hello World"
[INFO] [1722172962.125707732] [topic_helloworld_pub]: Publishing: "11_Hello World"
```

启动第二个 SSH 远程连接的终端，进入 aaa_ws 目录，设置路径，运行第二个节点

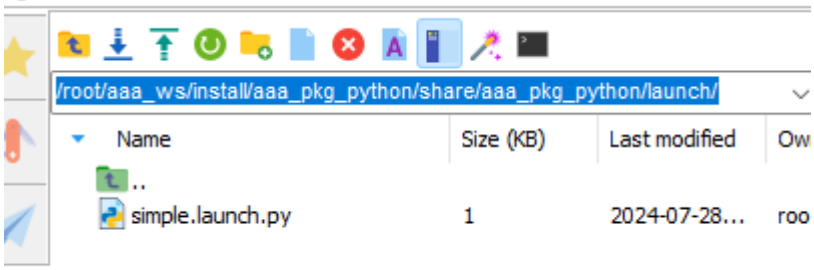
cd /userdata/aaa_ws
source install/setup.bash
ros2 run aaa_pkg_python sub

```
root@ubuntu:~# cd /userdata/aaa_ws
root@ubuntu:/userdata/aaa_ws# source install/setup.bash
root@ubuntu:/userdata/aaa_ws# ros2 run aaa_pkg_python sub
[INFO] [1735386071.754737030] [topic_helloworld_sub]: I heard: "81_Hello World"
[INFO] [1735386072.179480919] [topic_helloworld_sub]: I heard: "82_Hello World"
[INFO] [1735386072.685894877] [topic_helloworld_sub]: I heard: "83_Hello World"
[INFO] [1735386073.181539772] [topic_helloworld_sub]: I heard: "84_Hello World"
```

两个窗口都运行 ctr+c 把程序中断

4）运行 launch 文件

在 aaa_ws/install/aaa_pkg_python/share/aaa_pkg_python/目录下，新建文件夹 launch，上传 simple.launch.py 到这个文件夹



双击 simple.launch.py 文件，进行修改功能包和节点名

```
* simple.launch.py
1  from launch import LaunchDescription                    # launch文件的描述类
2  from launch_ros.actions import Node                     # 节点启动的描述类
3
4  def generate_launch_description():                      # 自动生成launch文件的函数
5      return LaunchDescription([                          # 返回launch文件的描述信息
6          Node(                                           # 配置一个节点的启动
7              package='aaa_pkg_python',       # 节点所在的功能包
8              executable='pub', # 节点的可执行文件
9          ),
10         Node(                                           # 配置一个节点的启动
11             package='aaa_pkg_python',       # 节点所在的功能包
12             executable='sub', # 节点的可执行文件名
13         ),
14     ])
15
```

运行

ros2 launch aaa_pkg_python simple.launch.py

```
root@ubuntu:~/aaa_ws# ros2 launch aaa_pkg_python simple.launch.py
[INFO] [launch]: All log files can be found below /root/.ros/log/2024-07-28-21-35-14-287251-ubuntu-3805
[INFO] [launch]: Default logging verbosity is set to INFO
[INFO] [pub-1]: process started with pid [3807]
[INFO] [sub-2]: process started with pid [3809]
[sub-2] [INFO] [1722173716.783774765] [topic_helloworld_sub]: I heard: "0_Hello World"
[pub-1] [INFO] [1722173716.786885388] [topic_helloworld_pub]: Publishing: "0_Hello World"
[pub-1] [INFO] [1722173717.209723485] [topic_helloworld_pub]: Publishing: "1_Hello World"
[sub-2] [INFO] [1722173717.210557326] [topic_helloworld_sub]: I heard: "1_Hello World"
[pub-1] [INFO] [1722173717.709744943] [topic_helloworld_pub]: Publishing: "2_Hello World"
[sub-2] [INFO] [1722173717.710549035] [topic_helloworld_sub]: I heard: "2_Hello World"
[pub-1] [INFO] [1722173718.209745118] [topic_helloworld_pub]: Publishing: "3_Hello World"
[sub-2] [INFO] [1722173718.210554918] [topic_helloworld_sub]: I heard: "3_Hello World"
[pub-1] [INFO] [1722173718.709741927] [topic_helloworld_pub]: Publishing: "4_Hello World"
[sub-2] [INFO] [1722173718.710536103] [topic_helloworld_sub]: I heard: "4_Hello World"
```

按 ctr+c 中断程序

思考题

根据 topic_helloworld_pub.py 程序，编程控制小车速度和转角

参数	修改前	修改后
话题（topic）	chatter	cmd_vel
数据格式	std_msgs/msg/string	geometry_msgs/msg/Twist
数值	msg = String() msg.data = str(self.count)+'_Hello World'	msg = Twist () msg. linear.x= msg. angular.z=

附

topic_helloworld_sub.py

```python
#!/usr/bin/env python3
# -*- coding: utf-8 -*-

import rclpy                                    # ROS2 Python 接口库
from rclpy.node import Node                     # ROS2 节点类
from std_msgs.msg import String                 # 字符串消息类型

"""
创建一个发布者节点
"""
class PublisherNode(Node):

    def __init__(self, name):
        super().__init__(name)                  # ROS2 节点父类初始化
        self.pub = self.create_publisher(String, "chatter", 10)   # 创建发布者
            对象（消息类型、话题名、队列长度）
```

```
        self.timer = self.create_timer(0.5, self.timer_callback)  # 创建一个定时器（单位为秒的周期，定时执行的回调函数）
        self.count=0

    def timer_callback(self):                   # 创建定时器周期执行的回调函数
        msg = String()                          # 创建一个 String 类型的消息对象
        msg.data = str(self.count)+'_Hello World'   # 填充消息对象中的消息数据
        self.pub.publish(msg)                   # 发布话题消息
        self.get_logger().info('Publishing: "%s"' % msg.data)     # 输出日志信息，提示已经完成话题发布
        self.count +=1

def main(args=None):                            # ROS2 节点主入口 main 函数
    rclpy.init(args=args)                       # ROS2 Python 接口初始化
    node = PublisherNode("topic_helloworld_pub") # 创建 ROS2 节点对象并进行初始化
    rclpy.spin(node)                            # 循环等待 ROS2 退出
    node.destroy_node()                         # 销毁节点对象
    rclpy.shutdown()                            # 关闭 ROS2 Python 接口
```

topic_helloworld_pub.py

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-

import rclpy                                    # ROS2 Python 接口库
from rclpy.node import Node                     # ROS2 节点类
from std_msgs.msg import String                 # 字符串消息类型

"""
创建一个发布者节点
"""
class PublisherNode(Node):

    def __init__(self, name):
        super().__init__(name)                          # ROS2 节点父类初始化
        self.pub = self.create_publisher(String, "chatter", 10)   # 创建发布者对象（消息类型、话题名、队列长度）
        self.timer = self.create_timer(0.5, self.timer_callback)  # 创建一个定时器（单位为秒的周期，定时执行的回调函数）
        self.count=0

    def timer_callback(self):                   # 创建定时器周期执行的回调函数
        msg = String()                          # 创建一个 String 类型的消息对象
        msg.data = str(self.count)+'_Hello World'   # 填充消息对象中的消息数据
```

```
        self.pub.publish(msg)                           # 发布话题消息
        self.get_logger().info('Publishing: "%s"' % msg.data)     # 输出日志信息，提示已经完成话题发布
        self.count +=1

def main(args=None):                                # ROS2 节点主入口 main 函数
    rclpy.init(args=args)                           # ROS2 Python 接口初始化
    node = PublisherNode("topic_helloworld_pub") # 创建 ROS2 节点对象并进行初始化
    rclpy.spin(node)                                # 循环等待 ROS2 退出
    node.destroy_node()                             # 销毁节点对象
    rclpy.shutdown()                                # 关闭 ROS2 Python 接口
```

simple.launch.py

```
from launch import LaunchDescription            # launch 文件的描述类
from launch_ros.actions import Node             # 节点启动的描述类

def generate_launch_description():              # 自动生成 launch 文件的函数
    return LaunchDescription([                  # 返回 launch 文件的描述信息
        Node(                                   # 配置一个节点的启动
            package='learning_pkg_python',      # 节点所在的功能包
            executable='pub', # 节点的可执行文件
        ),
        Node(                                   # 配置一个节点的启动
            package='learning_pkg_python',      # 节点所在的功能包
            executable='sub', # 节点的可执行文件名
        ),
    ])
```